Chapter 39 - Network Sockets

The shared socket device gives machine-language programmes an IPv4 TCP and UDP connection to a network. It is another card on the Intuition Engine bus: prepare a request in ordinary guest memory, point the device at it, issue one command, then read the result registers.

All six CPUs can use the same device. The four wide-address CPUs use the physical register block directly. The 6502 and Z80 use their Bank 3 window. BASIC does not map the socket device, so this chapter begins in IE Mon.

The first example does not contact another machine. It creates one UDP socket, checks the result, and closes it. A successful run paints the VideoChip raster band green and sounds an A. A failed run paints it red and sounds the lower A. That makes the result visible and audible without depending on a server.

39.1 Register block

The device occupies `$F2500` through `$F257F`. Its defined registers are `32` bits wide.

Address	Name	R/W	Meaning
`$F2500`	`HOST_SOCKET_CMD`	W	Command number. A complete write dispatches the request.
`$F2504`	`HOST_SOCKET_REQ_PTR`	R/W	Guest address of the request descriptor.
`$F2508`	`HOST_SOCKET_REQ_LEN`	R/W	Descriptor size in bytes. Use `96`.
`$F250C`	`HOST_SOCKET_RES1`	R	Primary result, or `$FFFFFFFF` on error.
`$F2510`	`HOST_SOCKET_RES2`	R	Secondary result.
`$F2514`	`HOST_SOCKET_ERRNO`	R	Socket error number.
`$F2518`	`HOST_SOCKET_HERRNO`	R	Name-resolution error number.
`$F251C`	`HOST_SOCKET_STATUS`	R	`0` ready, `1` busy, `2` error.
`$F2520`	`HOST_SOCKET_EVENTS`	R	Event word; currently reads `0`.

Commands execute synchronously in the current device. By the time the write to `HOST_SOCKET_CMD` returns, the result registers are ready. Status value `1` is defined for the interface but is not left visible by the current command path. Test `HOST_SOCKET_STATUS` and `HOST_SOCKET_ERRNO` after every command.

Addresses `$F2524` through `$F257F` are reserved. Do not use them as programme storage.

39.2 The request descriptor

Every command reads one `96`-byte descriptor containing `24` big-endian words. Clear the whole descriptor before filling the fields needed by a command. The same offsets have different roles for different commands.

Offset	Neutral name	Common use
`+$00`	`HOST_SOCKET_REQ_DOMAIN`	Address family, or socket descriptor.
`+$04`	`HOST_SOCKET_REQ_TYPE`	Socket type.

Offset	Neutral name	Common use
+$08	HOST_SOCKET_REQ_PROTOCOL	Protocol.
+$0C	HOST_SOCKET_REQ_DATA_PTR	Data, address, option, or name pointer.
+$10	HOST_SOCKET_REQ_DATA_LEN	Input length or output capacity.
+$14	HOST_SOCKET_REQ_ADDR_PTR	Address pointer or returned-length pointer.
+$18	HOST_SOCKET_REQ_ADDR_LEN	Address length.
+$1C	HOST_SOCKET_REQ_FLAGS	Send or receive flags.
+$20	HOST_SOCKET_REQ_LEVEL	Socket-option level.
+$24	HOST_SOCKET_REQ_OPTNAME	Socket-option name.
+$28	HOST_SOCKET_REQ_TIMEOUT_PTR	Pointer to an 8-byte timeout.
+$2C	HOST_SOCKET_REQ_SIGMASK_PTR	Selection signal mask value.
+$30	HOST_SOCKET_REQ_ARG1	Command-specific argument.
+$34	HOST_SOCKET_REQ_ARG2	Second command-specific argument.
+$38	HOST_SOCKET_REQ_EXCEPT_PTR	Exception descriptor-set pointer.
+$40	HOST_SOCKET_REQ_HOSTENT_NAME_PTR	Resolver output-name pointer.
+$44	HOST_SOCKET_REQ_HOSTENT_NAME_CAP	Resolver output-name capacity.
+$48	HOST_SOCKET_REQ_HOSTENT_ADDRS_PTR	Resolver IPv4 output pointer.
+$4C	HOST_SOCKET_REQ_HOSTENT_ADDRS_MAX	Maximum returned IPv4 addresses.

The remaining four words are reserved and must be zero.

The descriptor is big-endian even when the active CPU normally stores longwords little-endian. For example, domain value 2 is stored as bytes `00 00 00 02`. Socket payload bytes are not byte-swapped.

39.3 Command table

Value	Constant	Operation and principal fields
1	HOST_SOCKET_CMD_SOCKET	Create: domain, type, protocol. Returns a descriptor in RES1.
2	HOST_SOCKET_CMD_BIND	Bind: descriptor, address pointer, address length.
3	HOST_SOCKET_CMD_LISTEN	Listen: descriptor, backlog in ARG1.
4	HOST_SOCKET_CMD_ACCEPT	Accept: descriptor, output address pointer and capacity, returned-length pointer.
5	HOST_SOCKET_CMD_CONNECT	Connect: descriptor, address pointer, address length.
6	HOST_SOCKET_CMD_SENDTO	Send: descriptor, data pointer and length, flags, optional destination.
7	HOST_SOCKET_CMD_RECVFROM	Receive: descriptor, data pointer and capacity, flags, optional source output.
8	HOST_SOCKET_CMD_SHUTDOWN	Shutdown: descriptor and mode in ARG1.
9	HOST_SOCKET_CMD_SETSOCKOPT	Set option: descriptor, level, option name, data pointer and length.
10	HOST_SOCKET_CMD_GETSOCKOPT	Get option: descriptor, level, option name, output pointer and capacity.

Value	Constant	Operation and principal fields
11	HOST_SOCKET_CMD_GETSOCKNAME	Read local address into the supplied output area.
12	HOST_SOCKET_CMD_GETPEERNAME	Read peer address into the supplied output area.
13	HOST_SOCKET_CMD_IOCTL	Control: descriptor, operation in ARG1, argument pointer in DATA_PTR.
14	HOST_SOCKET_CMD_CLOSE	Close the descriptor in the first word.
15	HOST_SOCKET_CMD_WAITSELECT	Wait on read, write, and exception descriptor sets.
16	HOST_SOCKET_CMD_GETHOSTBYNAME	Resolve the zero-terminated name at DATA_PTR.
17	HOST_SOCKET_CMD_GETHOSTBYADDR	Reverse-resolve the four IPv4 bytes at DATA_PTR.
18	HOST_SOCKET_CMD_GETHOSTNAME	Write the machine name to DATA_PTR, bounded by DATA_LEN.
19	HOST_SOCKET_CMD_DUP2	Duplicate descriptor ARG1 as ARG2.
20	HOST_SOCKET_CMD_GETEVENTS	Read the event word, optionally also writing it to DATA_PTR; currently returns 0.
21	HOST_SOCKET_CMD_RELEASE	Release a descriptor under the key in ARG1.
22	HOST_SOCKET_CMD_RELEASECOPY	Release a duplicate while keeping the original descriptor active.
23	HOST_SOCKET_CMD_OBTAIN	Obtain a released descriptor by key.

An IPv4 socket-address record is 16 bytes. Byte 0 is length 16, byte 1 is family 2, bytes 2 and 3 are the big-endian port, bytes 4 through 7 are the four IPv4 octets, and bytes 8 through 15 are zero. Selection descriptor sets are exactly 8 bytes, giving the device its fixed 64-descriptor limit. A timeout is two big-endian signed words: seconds followed by microseconds. A null timeout pointer means no timeout.

39.4 Results, status, and errors

Successful integer-returning commands place their signed result in HOST_SOCKET_RES1. Successful status-only commands return 0 there. An error sets HOST_SOCKET_RES1 to $FFFFFFFF, HOST_SOCKET_STATUS to 2, and HOST_SOCKET_ERRNO to the error number. Name-resolution failures also use HOST_SOCKET_HERRNO.

The errors a small programme most often needs are:

Value	Meaning
9	Bad or unknown descriptor.
22	Invalid descriptor, pointer, length, or argument.
35	Operation would block, or a released key is not ready.
40	Message or requested transfer is too large.
45	Operation is not supported.
55	No buffer or descriptor capacity remains.
78	Network service unavailable.

Resolver error 1 means host not found. Resolver error 3 is defined as a non-recoverable resolver failure; the current resolver path reports its failures as 1.

The device supports IPv4 TCP and UDP. Raw sockets, packet filters, route manipulation, interface configuration, and monitoring operations are unsupported.

39.5 CPU access

IE64, IE32, M68K, and x86 use `$F2500` directly. The public include for each CPU provides the same `HOST_SOCKET_*` names.

The 6502 and Z80 select `HOST_SOCKET_BANK`, value `$79`, in Bank 3 and then use the `$6500` window. Their includes provide `HOST_SOCKET_SELECT` to perform the selection.

The Bank 3 register view is byte-wide and big-endian:

Window address	Longword lane
`$6500`	bits 31 through 24.
`$6501`	bits 23 through 16.
`$6502`	bits 15 through 8.
`$6503`	bits 7 through 0.

The same lane order applies to `REQ_PTR` and `REQ_LEN`. A command written through `$6500` through `$6503` dispatches only after all four lanes have been written. This prevents a partial command value from starting an operation.

39.6 First socket from IE Mon

Enter IE Mon with a non-BASIC IE64 programme active. First clear the request record at `$2000`, then store domain `2` and datagram type `2` as big-endian words:

```
(ie64)> f 2000 205F 00
(ie64)> w 2003 02
(ie64)> w 2007 02
```

Now enter the programme with IE64 assemble mode. The branch targets are literal because monitor assemble mode accepts one instruction at a time:

```
(ie64)> A 1000
asm $0000000000001000> move.q r20,#$2000
asm $0000000000001008> move.q r21,#$F2500
asm $0000000000001010> move.q r1,#$2000
asm $0000000000001018> store.l r1,4(r21)
asm $0000000000001020> move.q r1,#$60
asm $0000000000001028> store.l r1,8(r21)
asm $0000000000001030> move.q r1,#1
asm $0000000000001038> store.l r1,(r21)
asm $0000000000001040> load.l r3,12(r21)
asm $0000000000001048> load.l r4,20(r21)
asm $0000000000001050> bne r4,r0,$1090
asm $0000000000001058> bswap.l r5,r3
asm $0000000000001060> store.l r5,(r20)
asm $0000000000001068> move.q r1,#14
asm $0000000000001070> store.l r1,(r21)
asm $0000000000001078> move.q r6,#$C040
asm $0000000000001080> move.q r7,#$1B800
asm $0000000000001088> bra $10A0
asm $0000000000001090> move.q r6,#$40C0
asm $0000000000001098> move.q r7,#$DC00
asm $00000000000010A0> move.q r2,#$F0000
asm $00000000000010A8> move.q r1,#1
asm $00000000000010B0> store.l r1,(r2)
asm $00000000000010B8> move.q r1,#7
asm $00000000000010C0> store.l r1,4(r2)
asm $00000000000010C8> move.q r2,#$F0048
asm $00000000000010D0> store.l r0,(r2)
asm $00000000000010D8> move.q r1,#540
asm $00000000000010E0> store.l r1,4(r2)
asm $00000000000010E8> store.l r6,8(r2)
asm $00000000000010F0> move.q r1,#1
asm $00000000000010F8> store.l r1,12(r2)
asm $0000000000001100> move.q r2,#$F0800
asm $0000000000001108> store.b r1,(r2)
asm $0000000000001110> move.q r2,#$F0900
asm $0000000000001118> store.l r7,(r2)
asm $0000000000001120> move.q r1,#190
asm $0000000000001128> store.b r1,4(r2)
asm $0000000000001130> move.q r1,#2
asm $0000000000001138> store.b r1,8(r2)
asm $0000000000001140> bra $1140
asm $0000000000001148>
Exited IE64 assemble mode
```

Disassemble before running. This is also the byte proof for the programme:

```
(ie64)> d 1000 #41
  00001000: 01 A7 00 00 00 20 00 00  move.q r20, #$2000
  00001008: 01 AF 00 00 00 25 0F 00  move.q r21, #$F2500
  00001010: 01 0F 00 00 00 20 00 00  move.q r1, #$2000
  00001018: 11 0D A8 00 04 00 00 00  store.l r1, 4(r21)
  00001020: 01 0F 00 00 60 00 00 00  move.q r1, #$60
  00001028: 11 0D A8 00 08 00 00 00  store.l r1, 8(r21)
  00001030: 01 0F 00 00 01 00 00 00  move.q r1, #$1
  00001038: 11 0C A8 00 00 00 00 00  store.l r1, (r21)
  00001040: 10 1D A8 00 0C 00 00 00  load.l r3, 12(r21)
  00001048: 10 25 A8 00 14 00 00 00  load.l r4, 20(r21)
  00001050: 42 06 20 00 40 00 00 00  bnez r4, $001090
  00001058: 3D 2C 18 00 00 00 00 00  bswap.l r5, r3
  00001060: 11 2C A0 00 00 00 00 00  store.l r5, (r20)
  00001068: 01 0F 00 00 0E 00 00 00  move.q r1, #$E
  00001070: 11 0C A8 00 00 00 00 00  store.l r1, (r21)
  00001078: 01 37 00 00 40 C0 00 00  move.q r6, #$C040
  00001080: 01 3F 00 00 00 B8 01 00  move.q r7, #$1B800
  00001088: 40 06 00 00 18 00 00 00  bra $0010A0
  00001090: 01 37 00 00 C0 40 00 00  move.q r6, #$40C0
  00001098: 01 3F 00 00 00 DC 00 00  move.q r7, #$DC00
  000010A0: 01 17 00 00 00 00 0F 00  move.q r2, #$F0000
  000010A8: 01 0F 00 00 01 00 00 00  move.q r1, #$1
  000010B0: 11 0C 10 00 00 00 00 00  store.l r1, (r2)
  000010B8: 01 0F 00 00 07 00 00 00  move.q r1, #$7
  000010C0: 11 0D 10 00 04 00 00 00  store.l r1, 4(r2)
  000010C8: 01 17 00 00 48 00 0F 00  move.q r2, #$F0048
  000010D0: 11 04 10 00 00 00 00 00  store.l r0, (r2)
  000010D8: 01 0F 00 00 1C 02 00 00  move.q r1, #$21C
  000010E0: 11 0D 10 00 04 00 00 00  store.l r1, 4(r2)
  000010E8: 11 35 10 00 08 00 00 00  store.l r6, 8(r2)
  000010F0: 01 0F 00 00 01 00 00 00  move.q r1, #$1
  000010F8: 11 0D 10 00 0C 00 00 00  store.l r1, 12(r2)
  00001100: 01 17 00 00 00 08 0F 00  move.q r2, #$F0800
  00001108: 11 08 10 00 00 00 00 00  store.b r1, (r2)
  00001110: 01 17 00 00 00 09 0F 00  move.q r2, #$F0900
  00001118: 11 3C 10 00 00 00 00 00  store.l r7, (r2)
  00001120: 01 0F 00 00 BE 00 00 00  move.q r1, #$BE
  00001128: 11 09 10 00 04 00 00 00  store.b r1, 4(r2)
  00001130: 01 0F 00 00 02 00 00 00  move.q r1, #$2
  00001138: 11 09 10 00 08 00 00 00  store.b r1, 8(r2)
T 00001140: 40 06 00 00 00 00 00 00  bra $001140
```

If you did not use assemble mode, enter the same bytes directly. This form is longer, but it is independent of the assembler and gives the monitor harness an exact runnable copy:

```
(ie64)> f 2000 205F 00
Filled $2000-$205F with $00
(ie64)> w 2003 02
(ie64)> w 2007 02
(ie64)> w 1000 01 A7 00 00 00 20 00 00 01 AF 00 00 00 25 0F 00
(ie64)> w 1010 01 0F 00 00 00 20 00 00 11 0D A8 00 04 00 00 00
(ie64)> w 1020 01 0F 00 00 60 00 00 00 11 0D A8 00 08 00 00 00
(ie64)> w 1030 01 0F 00 00 01 00 00 00 11 0C A8 00 00 00 00 00
(ie64)> w 1040 10 1D A8 00 0C 00 00 00 10 25 A8 00 14 00 00 00
(ie64)> w 1050 42 06 20 00 40 00 00 00 3D 2C 18 00 00 00 00 00
(ie64)> w 1060 11 2C A0 00 00 00 00 00 01 0F 00 00 0E 00 00 00
(ie64)> w 1070 11 0C A8 00 00 00 00 00 01 37 00 00 40 C0 00 00
(ie64)> w 1080 01 3F 00 00 00 B8 01 00 40 06 00 00 18 00 00 00
(ie64)> w 1090 01 37 00 00 C0 40 00 00 01 3F 00 00 00 DC 00 00
(ie64)> w 10A0 01 17 00 00 00 00 0F 00 01 0F 00 00 01 00 00 00
(ie64)> w 10B0 11 0C 10 00 00 00 00 00 01 0F 00 00 07 00 00 00
(ie64)> w 10C0 11 0D 10 00 04 00 00 00 01 17 00 00 48 00 0F 00
(ie64)> w 10D0 11 04 10 00 00 00 00 00 01 0F 00 00 1C 02 00 00
(ie64)> w 10E0 11 0D 10 00 04 00 00 00 11 35 10 00 08 00 00 00
(ie64)> w 10F0 01 0F 00 00 01 00 00 00 11 0D 10 00 0C 00 00 00
(ie64)> w 1100 01 17 00 00 00 08 0F 00 11 08 10 00 00 00 00 00
(ie64)> w 1110 01 17 00 00 00 09 0F 00 11 3C 10 00 00 00 00 00
(ie64)> w 1120 01 0F 00 00 BE 00 00 00 11 09 10 00 04 00 00 00
(ie64)> w 1130 01 0F 00 00 02 00 00 00 11 09 10 00 08 00 00 00
(ie64)> w 1140 40 06 00 00 00 00 00 00
(ie64)> r pc 1000
(ie64)> b 1140
(ie64)> g
```

The byte-entry form runs to the final loop. Use the disassembly above to check each row before `g`, then inspect the socket registers:

```
(ie64)> m F2500 3
```

On success, the close command leaves status `0`, result `0`, and error `0`. The raster band is green and the square voice sounds at `440` Hz. On failure, status is `2`, result is `$FFFFFFFF`, and the error word explains the failure. The raster band is red and the voice sounds at `220` Hz.

Lines `$1000` through `$1038` stage and issue `SOCKET`. Lines `$1040` through `$1050` collect the returned descriptor and error. The descriptor in `R3` is a native IE64 value, so `BSWAP.L` converts it before the close request stores it into the big-endian descriptor. The remaining lines turn the result into an immediate picture and sound.

39.7 Sending and receiving

For `SENDTO`, place the socket descriptor in the first word, a payload pointer and length at `+$0C` and `+$10`, flags at `+$1C`, and an optional destination pointer and length at `+$14` and `+$18`. `HOST_SOCKET_RES1` returns the byte count.

For `RECVFROM`, the data length is a capacity. The command refuses a capacity above `65536`. On success it writes received bytes to the data pointer and returns their count. An optional source-address area uses the address pointer and capacity, with its returned length written through the pointer in `ARG1`.

Sockets are non-blocking. Error `35` means that the requested work would block. Use `WAITSELECT` to wait for readiness rather than spinning continuously.

39.8 Cleanup and limits

Close every descriptor that the programme no longer needs. Descriptors are transient programme state, not persistent data.

The fixed limits are:

Item	Limit
Request descriptor	`96` bytes.
One send or receive	`65536` bytes.
One socket address	`128` bytes.
Guest descriptor table	`64` entries.
Selection descriptor set	`8` bytes.
Resolver input name	`255` bytes including its terminator limit.

Keep request records, payloads, address records, descriptor sets, and returned-length words in mapped guest RAM for the whole command. Invalid spans fail with error `22`; the device does not partially use an invalid request.